

Universidade Estadual da Paraíba – UEPB

Laboratório de Programação II

Projeto da Unidade 2

1 Diretrizes gerais

O projeto deverá apresentar um sistema desenvolvido em Java, com tema escolhido pelo grupo e funcionalidades coerentes com o problema proposto. Os conteúdos obrigatórios deverão participar efetivamente do funcionamento do sistema, não sendo suficiente criar classes ou métodos apenas para cumprir formalmente os requisitos.

Os grupos poderão ser formados por **1, 2, 3 ou 4 estudantes**. Na apresentação, **um integrante será sorteado** para demonstrar e explicar o projeto. Todos os integrantes deverão conhecer integralmente o código entregue.

2 Requisitos obrigatórios

O projeto deverá conter:

- encapsulamento, com atributos privados e alterações realizadas por métodos adequados;
- pelo menos **duas coleções de tipos distintos**, como `ArrayList`, `HashMap` ou `HashSet`;
- herança com uma **classe abstrata** e pelo menos duas classes filhas concretas;
- polimorfismo de herança, utilizando objetos das classes filhas pelo tipo da classe abstrata;
- pelo menos uma **interface**, implementada por duas ou mais classes;
- polimorfismo de interface, utilizando os objetos pelo tipo da interface;
- pelo menos um **enum** utilizado em atributos, parâmetros, retornos ou regras do sistema;
- tratamento adequado de exceções em pelo menos **duas situações diferentes**;
- operações de cadastro, busca ou consulta e alteração de dados;
- regras de negócio distribuídas entre as classes responsáveis.

Não será considerado válido criar recursos que não sejam utilizados nas operações demonstradas. Também deverão ser evitados atributos públicos, repetição de código, métodos excessivamente longos, condicionais ou `instanceof` utilizados para substituir o polimorfismo e blocos `catch` vazios.

3 Organização mínima do projeto

O projeto deverá possuir, no mínimo:

- uma classe `Main` limpa, responsável apenas por iniciar o sistema;
- uma classe responsável pela leitura e saída de dados, localizada entre a `Main` e o controlador;
- uma classe controladora, responsável por receber as solicitações e delegar as operações;
- classes de domínio responsáveis pelos dados e pelas regras do sistema.

A classe de leitura e saída deverá concentrar o uso de `Scanner` e `System.out`. O controlador deverá apenas receber as solicitações e delegar as operações para uma classe de sistema, ou outra classe responsável pelas regras de negócio, sem executar qualquer lógica diretamente.

4 Demonstração obrigatória

Durante a apresentação, o estudante sorteado deverá:

1. explicar brevemente o problema e as funcionalidades do sistema;
2. executar o projeto e demonstrar suas principais operações;
3. indicar onde cada conteúdo obrigatório foi aplicado;
4. justificar a escolha das duas coleções;
5. demonstrar o polimorfismo da classe abstrata;
6. demonstrar o polimorfismo da interface;
7. mostrar a utilização do `enum`;
8. demonstrar pelo menos duas situações de exceção;
9. responder às perguntas sobre qualquer parte do código.

5 O que será considerado na nota

A avaliação considerará:

- atendimento aos requisitos obrigatórios;
- funcionamento correto das funcionalidades apresentadas;
- integração coerente entre herança, interface, coleções, `enum` e exceções;
- organização, legibilidade e divisão de responsabilidades do código;
- domínio do estudante sobre o projeto durante a apresentação;
- capacidade de explicar as decisões adotadas e responder às perguntas.

A inclusão de testes de unidade com **JUnit** poderá acrescentar **0,5 ponto** à nota, desde que os testes sejam executados e o estudante explique os casos elaborados, os resultados esperados e o que está sendo verificado.

Atenção: caso a apresentação seja considerada insuficiente, será aplicada uma redução de **70% sobre a nota obtida no projeto**. A apresentação será insuficiente quando o projeto não funcionar, os requisitos não puderem ser demonstrados ou o estudante sorteado não conseguir explicar o código e responder às perguntas.

6 Exemplo esperado para a classe Main

A classe **Main** deverá permanecer limpa e ser responsável apenas por iniciar a execução do programa. Ela não deverá conter menus, leituras de dados, impressões, validações ou regras de negócio.

Um exemplo de organização esperada é:

```
public class Main {  
  
    public static void main(String[] args) {  
        SistemaConsole sistemaConsole = new SistemaConsole();  
        sistemaConsole.iniciar();  
    }  
}
```

Toda a interação com o usuário deverá começar pelo método `iniciar()` da classe `SistemaConsole`. Essa classe deverá concentrar o uso de `Scanner` e `System.out`, comunicando-se com o controlador para solicitar as operações do sistema.

7 Estrutura esperada do projeto

A figura a seguir apresenta a organização mínima esperada para o projeto. A classe **Main** deverá apenas iniciar o sistema. A classe **SistemaConsole** deverá concentrar a interação com o usuário, incluindo o uso de **Scanner** e **System.out**. O controlador deverá receber as solicitações e delegá-las para a classe responsável pela lógica do sistema.

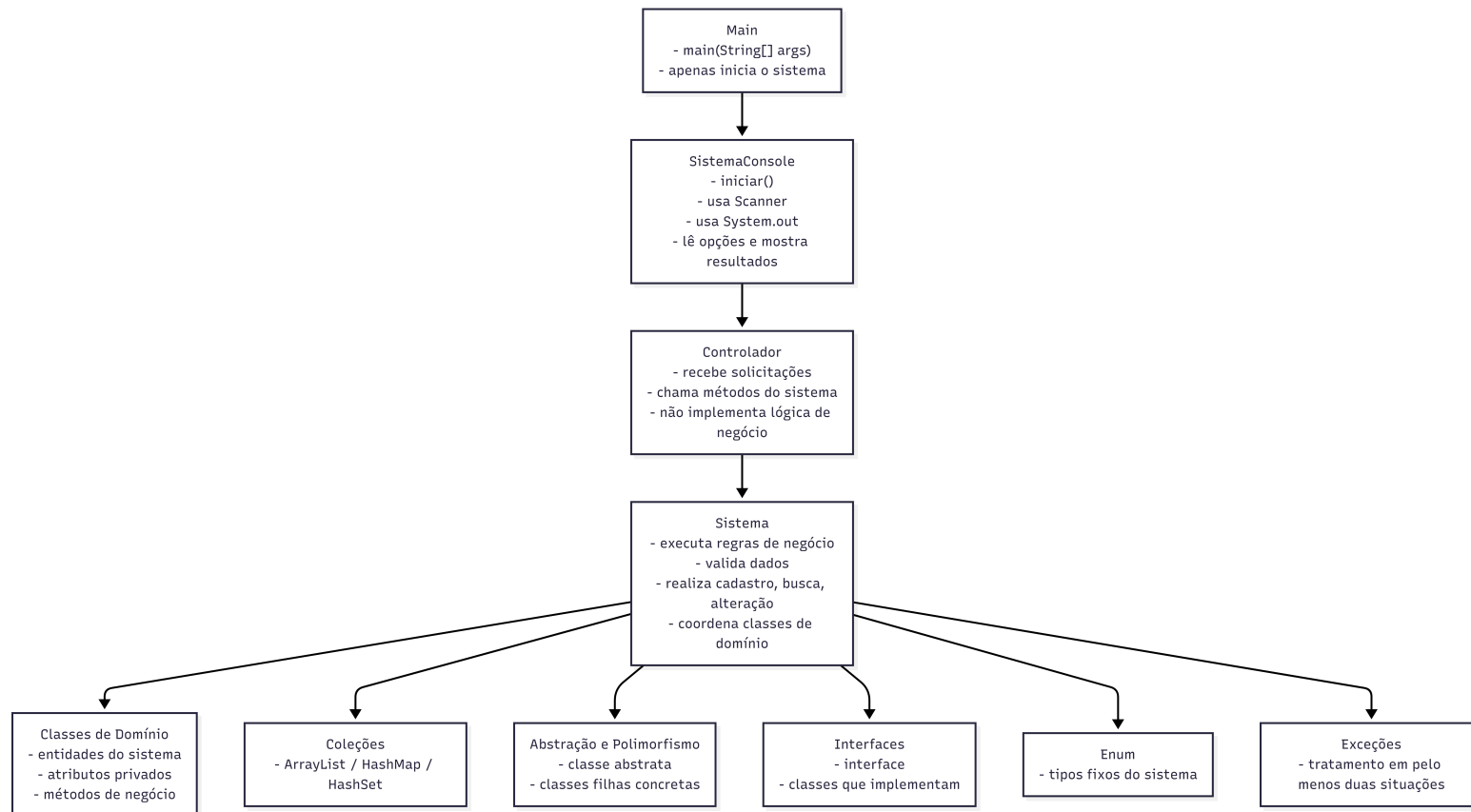


Figura 1 – Estrutura esperada para a organização do projeto.

A classe responsável pelo sistema deverá coordenar as classes de domínio e concentrar as operações gerais, como cadastro, busca, consulta e alteração. As regras específicas de cada entidade deverão permanecer nas classes responsáveis por essas regras.